

Documentation du Script de Migration sous OpenStack (Python)

Introduction

Ce script a pour objectif d'orchestrer la migration d'une machine virtuelle (VM) dans un environnement OpenStack, en effectuant une série de vérifications (compatibilité Xen, réseau, CPU, et règles de migration définies dans un fichier JSON). Le script utilise plusieurs outils et modules (comme OpenStack CLI, CSV, JSON, et SSH) pour récupérer les informations nécessaires et valider la migration avant de la lancer. Il s'intègre dans une chaîne d'outils permettant de migrer des VMs en live d'un cluster à un autre.

Contexte d'utilisation :

- Le script s'exécute sur un environnement de contrôle (controller) d'OpenStack.
- Il repose sur un fichier CSV (`/etc/node-clusters.csv`) qui contient des informations sur les clusters (nom, VLAN, version Xen, flags CPU).
- Un fichier JSON (`/etc/openstack/migration_rules.json`) définit les règles de migration entre clusters.
- La migration est déclenchée via la commande `openstack server migrate --live` si toutes les vérifications préalables sont validées.

Pour en savoir plus sur l'orchestration des migrations sous OpenStack, consultez la [documentation officielle OpenStack CLI](#).

Contenu et Explication du Code

1. Importations et Configuration du Logging

```
import csv
```

```
import json
```

```
import subprocess
```

```
import logging
```

```
import argparse
```

from typing import Dict, Any

- **Modules utilisés :**

- `csv` et `json` pour la lecture et le traitement des fichiers de configuration.
- `subprocess` pour exécuter des commandes système (comme les commandes OpenStack ou SSH).
- `logging` pour la journalisation des opérations et erreurs.
- `argparse` pour la gestion des arguments de la ligne de commande.
- `typing` pour les annotations de type.

```
logging.basicConfig(  
    level=logging.INFO,  
    format='%(asctime)s - %(levelname)s - %(message)s',  
    filename='openstack_migration.log'  
)
```

- **Configuration du logging :**

- Niveau d'information défini à `INFO` pour enregistrer les informations utiles.
- Format incluant la date, le niveau et le message.
- Les logs sont enregistrés dans le fichier `openstack_migration.log`.

Référence : [Python Logging HOWTO](#)

2. Fonction `get_vm_host`

```
def get_vm_host(vm_id: str) -> str:
```

```
    """Récupère l'hôte actuel d'une VM via OpenStack CLI"""
```

```

cmd = [
    'openstack', 'server', 'show', vm_id,
    '-c', 'OS-EXT-SRV-ATTR:hypervisor_hostname',
    '-f', 'value'
]

try:
    result = subprocess.run(cmd, check=True, capture_output=True, text=True)
    return result.stdout.strip()

except subprocess.CalledProcessError as e:
    logging.error(f"Erreur lors de la récupération de l'hôte : {e.stderr}")
    raise

```

- **But** : Récupérer le nom de l'hôte (hyperviseur) sur lequel la VM est actuellement déployée.
- **Détails** :
 - La commande OpenStack CLI `openstack server show` est utilisée pour extraire l'attribut `OS-EXT-SRV-ATTR:hypervisor_hostname`.
 - En cas d'erreur, celle-ci est loguée et l'exception est relancée pour interrompre le processus.

Référence : [OpenStack Server Show CLI](#)

3. Fonction `get_cluster_details`

```

def get_cluster_details(host: str) -> Dict[str, Any]:
    """Récupère les détails du cluster depuis le CSV enrichi"""

    with open('/etc/node-clusters.csv') as f:
        reader = csv.DictReader(f)

        for row in reader:

```

```

if row['hostname'] == host:

    return {

        'cluster': row['cluster'],

        'vlan': row['vlan'],

        'xen_version': row['xen_version'],

        'cpu_flags': row['cpu_flags'].split(';')

    }

raise ValueError(f"Hôte {host} non trouvé dans le CSV")

```

- **But** : Lire le fichier CSV pour obtenir les détails associés à un hôte donné.
- **Fonctionnement** :
 - Utilisation de `csv.DictReader` pour traiter le fichier `/etc/node-clusters.csv`.
 - Si l'hôte correspondant est trouvé, renvoie un dictionnaire contenant :
 - `cluster` : Nom du cluster.
 - `vlan` : VLAN associé.
 - `xen_version` : Version de Xen utilisée.
 - `cpu_flags` : Liste des flags CPU (transformés en liste avec `split(';')`).
 - En cas de non-correspondance, une exception est levée.

Conseil : Assurez-vous que le CSV est correctement formaté et que toutes les colonnes nécessaires sont présentes.

4. Fonction `check_xen_compatibility`

```
def check_xen_compatibility(source_host: str, target_host: str) -> bool:
```

```
    """Vérifie la compatibilité Xen via SSH"""
```

```
    try:
```

```

# Récupération des informations Xen

cmd_source = f"ssh {source_host} xl info | grep -E 'xen_version|cc_compiler'"

cmd_target = f"ssh {target_host} xl info | grep -E 'xen_version|cc_compiler'"

source_info = subprocess.run(cmd_source, shell=True, capture_output=True,
text=True)

target_info = subprocess.run(cmd_target, shell=True, capture_output=True, text=True)

return source_info.stdout == target_info.stdout

except Exception as e:

    logging.error(f"Échec vérification Xen : {str(e)}")

    return False

```

- **But** : Comparer les informations Xen (version et compilateur) entre l'hôte source et l'hôte cible.
- **Détails** :
 - Les commandes SSH exécutent **xl info** sur chaque hôte pour récupérer des détails sur Xen, filtrés par **grep**.
 - La compatibilité est validée si les sorties des deux commandes sont identiques.
 - En cas d'erreur, la fonction logue l'erreur et retourne **False**.

Référence : [Xen Hypervisor Documentation](#)

5. Fonction **check_network_compatibility**

```

def check_network_compatibility(source_vlan: str, target_vlan: str) -> bool:

    """Vérifie la connectivité réseau entre VLANs"""

    cmd = f"openstack network show --vlan {source_vlan} -c subnets -f value"

```

try:

```
result = subprocess.run(cmd.split(), capture_output=True, text=True)
```

```
source_subnet = result.stdout.strip()
```

```
return source_subnet in get_routed_subnets(target_vlan)
```

except Exception as e:

```
logging.error(f"Échec vérification réseau : {str(e)}")
```

```
return False
```

- **But** : Vérifier que les sous-réseaux (subnets) associés au VLAN source sont accessibles depuis le VLAN cible.
- **Détails** :
 - Exécute la commande `openstack network show` pour obtenir les sous-réseaux du VLAN source.
 - La fonction `get_routed_subnets(target_vlan)` (supposée être définie ailleurs) doit retourner les sous-réseaux routés pour le VLAN cible.
 - Retourne `True` si le sous-réseau source est présent dans les sous-réseaux du VLAN cible, sinon `False`.

Conseil : Vérifier l'implémentation ou la disponibilité de la fonction `get_routed_subnets` pour éviter des erreurs d'exécution.

6. Fonction `migrate_vm`

```
def migrate_vm(vm_id: str, target_host: str, dry_run: bool = False) -> None:
```

```
    """Fonction principale de migration avec vérifications"""
```

try:

```
    # Récupération des informations
```

```
    source_host = get_vm_host(vm_id)
```

```
    source_details = get_cluster_details(source_host)
```

```
    target_details = get_cluster_details(target_host)
```

```

# Chargement des règles de migration

with open('/etc/openstack/migration_rules.json') as f:

    migration_rules = json.load(f)


# Vérification multi-couches

checks = {

    'json_rule': migration_rules.get(source_details['cluster'], {})

        .get(target_details['cluster'], {}).get('success', False),

    'xen': check_xen_compatibility(source_host, target_host),

    'network': check_network_compatibility(

        source_details['vlan'],

        target_details['vlan']

    ),

    'cpu': set(source_details['cpu_flags']).issubset(

        set(target_details['cpu_flags'])

    )

}


# Logique de décision

if all(checks.values()):

    if dry_run:

        logging.info(f"DRY RUN: Migration de {vm_id} vers {target_host} possible")

        return

    cmd = [

```

```

        'openstack', 'server', 'migrate',
        '--live', target_host, vm_id, '--wait'
    ]

    subprocess.run(cmd, check=True)

    logging.info(f"Migration réussie de {vm_id} vers {target_host}")

```

else:

```

    failed = [k for k, v in checks.items() if not v]

    msg = f"Migration bloquée - Échec: {' '.join(failed)}"

    logging.warning(msg)

    print(msg)

```

except Exception as e:

```

    logging.error(f"Erreur critique: {str(e)}")

    raise

```

- **But :** Orchestrer l'ensemble du processus de migration en effectuant plusieurs vérifications avant de déclencher la migration live.
- **Étapes détaillées :**
 1. **Récupération des informations :**
 - `get_vm_host(vm_id)` permet d'obtenir l'hôte source de la VM.
 - `get_cluster_details` récupère les détails (cluster, VLAN, version Xen, flags CPU) pour l'hôte source et l'hôte cible.
 2. **Chargement des règles de migration :**
 - Lecture du fichier JSON (`/etc/openstack/migration_rules.json`) qui définit les règles entre clusters.

3. Vérifications multi-couches :

- **json_rule** : Vérifie que la règle dans le fichier JSON autorise la migration entre le cluster source et le cluster cible.
- **xen** : Vérifie la compatibilité des versions Xen entre les hôtes.
- **network** : Vérifie que les VLANs (et donc les sous-réseaux) sont compatibles.
- **cpu** : S'assure que les flags CPU requis sur le source sont bien présents sur le target.

4. Logique de décision :

- Si toutes les vérifications sont réussies (`all(checks.values())`), la migration est lancée.
- En mode `dry_run`, le script se contente de logger que la migration est possible sans l'exécuter.
- Sinon, la migration est effectuée avec la commande OpenStack.
- En cas d'échec d'une ou plusieurs vérifications, la migration est bloquée et un message d'erreur est affiché.

Référence : Pour approfondir la gestion de la migration avec OpenStack, consultez [la documentation OpenStack sur la migration live](#).

7. Bloc Principal et Gestion des Arguments

```
if __name__ == "__main__":
```

```
    parser = argparse.ArgumentParser()
```

```
    parser.add_argument('vm_id', help='ID de la VM à migrer')
```

```
    parser.add_argument('target_host', help='Hôte de destination')
```

```
    parser.add_argument('--dry-run', action='store_true', help='Simulation')
```

```
    args = parser.parse_args()
```

```
    migrate_vm(args.vm_id, args.target_host, args.dry_run)
```

- **But :** Permettre l'exécution du script via la ligne de commande.
- **Détails :**
 - Utilisation d'`argparse` pour définir et récupérer les arguments : `vm_id`, `target_host` et l'option `--dry-run` pour simuler la migration.
 - Appel de la fonction `migrate_vm` avec les arguments fournis.

Référence : [Argparse — Parser for command-line options, arguments and sub-commands](#)

- [Documentation OpenStack CLI](#)
- [Python subprocess module](#)
- [Python CSV module](#)
- [Python Logging HOWTO](#)
- [Argparse Documentation](#)